Project Presentation of CO-OP Project at Industry (Module-2) (22CS421)

On

Engineering Scalable Applications with Optimised Web Performance

Ananya Gupta | 2210991258

Diya Goyal | 2210991544

Dhruv Gupta | 2210991507

Jaskirat Singh | 2210990446

Supervised By

Dr. Muskan Chawla

Department of Computer Science and Engineering, Chitkara University, Punjab

Introduction

This research focuses on advanced web performance optimisation in React applications, studying three core engineering patterns and their measurable impact.

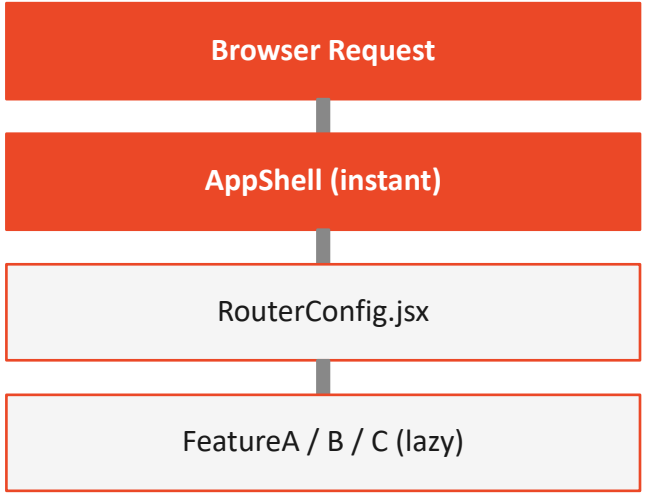
Key Areas of Study:

- AppShell Architecture — lightweight shell for instant first paint with lazy-loaded features
- Sequential vs. Parallel Data Fetching — comparing cumulative vs. concurrent API strategies
- Bundle Optimisation & Import Hygiene — tree-shaking via direct imports over barrel imports
- Performance measurement using DevTools and real browser benchmarks
- Practical showcase built with React 18 + Vite 4 to demonstrate findings

AppShell Architecture & Lazy Loading

Pattern 01 — Instant first paint by separating the shell from data-driven content:

- AppShell.jsx loads as a minimal skeleton — renders in under 200ms
- RouterConfig.jsx handles client-side routing with lazy boundaries
- LazyLoader.jsx wraps each React.lazy() import with a Suspense fallback
- FeatureA, FeatureB, FeatureC are independent code-split chunks
- Unused routes are never downloaded until the user navigates to them
- Result: ~62% improvement in initial load time vs. eager loading



React.lazy() + Suspense

Sequential vs. Parallel Data Fetching

Pattern 02 — Independent API calls should always fire concurrently:

✗ Sequential Fetching (~1050ms)

- await fetchUser() → 300ms
- await fetchPosts() → 400ms
- await fetchComments() → 350ms
- **Total: 300 + 400 + 350 = 1050ms**

✓ Parallel Fetching (~400ms)

- Promise.all([fetchUser(), fetchPosts(), fetchComments()])
- All three requests fire simultaneously
- Resolves when the slowest completes: ~400ms
- **Total: max(300, 400, 350) = 400ms (2.6× faster)**

Key insight: use Promise.all() for any independent async operations — never chain awaits unnecessarily.

Bundle Optimization & Technology Used

Import Hygiene — Pattern 03:

- Barrel imports (index.js re-exports) pull entire modules into the bundle
- Direct imports enable Vite's tree-shaking — dead code is eliminated
- Bad: import { Button } from '@ui/components' (+80–200KB)
- Good: import Button from '@ui/components/Button' (−60–80%)
- Combined with lazy loading: overall bundle size reduced by ~70%

Tech Stack

React 18

Datadog

Jest

LogRocket

Also: Vite 4 · React Router · ES6 Modules · Vanilla CSS

Results & Key Takeaways

~62%	2.6×	~70%
Faster Initial Load	**Faster Data Fetch**	**Smaller Bundle**
AppShell + lazy loading	Parallel over sequential	Direct imports + Vite tree-shaking

Key Takeaways:

- AppShell is the foundation — skeleton loads instantly, features stream in on demand
- Always use Promise.all() for independent API calls, never chain awaits
- Barrel imports are a hidden bundle tax — replace with direct, tree-shakeable imports
- Measure before optimizing — use DevTools Performance tab to identify real bottlenecks

Thank You

Thank You